

UD 00

Actividades evaluables 02

Ángel Esteban Parra

Actividad 01	2
Actividad 02	4
Sudoku.in	7
Actividad 03	8
Actividad 04	10
Actividad 05	13
Actividad 06	21
Alumno 2	22
Profesor 2	24
Escola 2	26
Código main	28
Actividad 07	33
Tests Actividad 07	43
Actividad 08	44
Actividad 09	48
Actividad 10	51
Actividad 11	53
Actividad 12	60
Actividad 13	63
Actividad 14	65
Actividad 15	68
Enlace GitHub	74

Actividad 01

```
1  # Angel Esteban Parra
2
3  # Actividad 01
4
5  # 1. FUNCIÓN LAMBDA
6
7  # Una función lambda es una función pequeña y sencilla
8  # que se puede escribir en una sola línea.
9
10 # En este ejemplo, creamos una función que recibe un número
11 # y devuelve su cuadrado.
12
13 cuadrado = lambda x: x ** 2
14
15 # Llamamos a la función lambda con el número 5.
16 resultado = cuadrado(5)
17
18 # Mostramos el resultado por pantalla.
19 print("El cuadrado de 5 es:", resultado)
20
21
22 # 2. CONVERTIR UNA CADENA DE TEXTO EN UNA LISTA DE ENTEROS
23
24 # Pedimos al usuario que introduzca varios números
25 # separados por espacios.
26
27 # Por ejemplo:
28 # 2 5 8 12 15 20
29
30 texto = input("Introduce varios números separados por espacios: ")
31
32 # La función split() separa la cadena cada vez que encuentra
33 # un espacio.
34
35
36 # Si el usuario escribe:
37 # "2 5 8 12"
38
39 # split() devuelve:
40 # ["2", "5", "8", "12"]
41
42 numeros_texto = texto.split()
43
44 # Ahora utilizamos map() junto con int().
45
46 # map(int, numeros_texto) convierte cada elemento de la lista
47 # de texto en un número entero.
48
49 # Por ejemplo:
50 # ["2", "5", "8"] -> [2, 5, 8]
51
52 # Como map() devuelve un objeto iterable, utilizamos list()
53 # para convertirlo en una lista.
54
55 numeros = list(map(int, numeros_texto))
56
57 # Mostramos la lista de números enteros.
58 print("Lista de números:", numeros)
59
60
61 # 3. ELIMINAR LOS NÚMEROS MENORES QUE 10
62
63 # Utilizamos filter() para quedarnos solamente con los
64 # números que cumplen una determinada condición.
65
```

```

66 # La función lambda comprueba si cada número es mayor o igual
67 # que 10.
68
69 # Si el número es >= 10, filter() lo conserva.
70 # Si es menor que 10, lo elimina.
71
72 numeros_mayores_igual_10 = list(
73     filter(lambda x: x >= 10, numeros)
74 )
75
76 # Mostramos la lista después de aplicar filter().
77 print("Números mayores o iguales que 10:",
78       numeros_mayores_igual_10)
79
80
81 # 4. MULTIPLICAR TODOS LOS ELEMENTOS CON reduce()
82
83 # Para utilizar reduce(), tenemos que importarlo desde
84 # el módulo functools.
85
86 from functools import reduce
87
88
89 # reduce() aplica una función a los elementos de una lista
90 # acumulando el resultado.
91
92 # En este caso:
93
94 # [10, 12, 20]
95
96 # Primero:
97 # 10 * 12 = 120
98

```

```

99 # Después:
100 # 120 * 20 = 2400
101
102 # Por tanto, el resultado final es 2400.
103
104 resultado_multiplicacion = reduce(
105     lambda x, y: x * y,
106     numeros_mayores_igual_10
107 )
108
109 # Mostramos el resultado final.
110 print("Multiplicación de los elementos:", resultado_multiplicacion)
111

```

Resultado:

```

El cuadrado de 5 es: 25
Introduce varios números separados por espacios: 20 30 40 1 1 2 3 4
Lista de números: [20, 30, 40, 1, 1, 2, 3, 4]
Números mayores o iguales que 10: [20, 30, 40]
Multiplicación de los elementos: 24000

```

Actividad 02

```
114 # Actividad 02
115
116 # COMPROBADOR DE SUDOKU
117
118 # Esta función recibe como parámetro el Sudoku.
119 # El Sudoku estará almacenado como una lista de listas.
120
121 # La función devolverá:
122 # True si el Sudoku es correcto
123 # False si el Sudoku es incorrecto
124
125 def esSudokuCorrecto(miArrayBi):
126
127     # 1. COMPROBAR LAS FILAS
128
129     # Recorremos las 9 filas del Sudoku.
130     for fila in miArrayBi:
131
132         # Convertimos la fila en un conjunto (set).
133         # Un set no permite elementos repetidos.
134
135         # Si una fila es:
136         # [5, 3, 4, 6, 7, 8, 9, 1, 2]
137
138         # tendrá 9 elementos diferentes.
139
140         # Si hay un número repetido, tendrá menos de 9.
141         if len(set(fila)) != 9:
142             return False
143
```

```
    # 2. COMPROBAR LAS COLUMNAS
144
145     # Recorremos las 9 columnas.
146     for columna in range(9):
147
148         # Creamos una lista vacía para guardar
149         # los números de la columna.
150         numeros_columna = []
151
152         # Recorremos las 9 filas.
153         for fila in range(9):
154
155             # Añadimos a la lista el número que está
156             # en la columna que estamos comprobando.
157             numeros_columna.append(miArrayBi[fila][columna])
158
159         # Comprobamos que no haya números repetidos.
160         if len(set(numeros_columna)) != 9:
161             return False
162
```

```

# 3. COMPROBAR LOS CUADRADOS 3x3

# Un Sudoku tiene 9 cuadrados de 3x3.

# Las variables filaInicio y columnaInicio indican
# dónde empieza cada cuadrado.

for filaInicio in range(0, 9, 3):
    for columnaInicio in range(0, 9, 3):
        # Lista donde guardaremos los números
        # del cuadrado 3x3.
        numeros_cuadrado = []

        # Recorremos las 3 filas del cuadrado.
        for fila in range(filaInicio, filaInicio + 3):
            # Recorremos las 3 columnas del cuadrado.
            for columna in range(
                columnaInicio,
                columnaInicio + 3
            ):
                # Guardamos el número.
                numeros_cuadrado.append(
                    miArrayBi[fila][columna]
                )

```

```

        # Comprobamos que los 9 números sean diferentes.
        if len(set(numeros_cuadrado)) != 9:
            return False

# 4. SI ESTÁ BIEN

# Si hemos llegado hasta aquí significa que:

# Las filas son correctas.
# Las columnas son correctas.
# Los cuadrados 3x3 son correctos.

# Por tanto, el Sudoku es correcto.

return True

```

```

211
212 # LEER EL ARCHIVO Sudoku.in
213
214 # Abrimos el archivo "Sudoku.in" en modo lectura ("r").
215 with open("Sudoku.in", "r") as archivo:
216
217     # Creamos una lista vacía donde guardaremos
218     # todo el Sudoku.
219     miArrayBi = []
220
221     # Leemos el archivo línea por línea.
222     for linea in archivo:
223
224         # Quitamos los espacios y el salto de línea
225         # que hay al final de cada línea.
226         linea = linea.strip()
227
228         # Separamos los números utilizando el espacio.
229
230         # Por ejemplo:
231         # "5 3 4 6 7 8 9 1 2"
232
233         # se convierte en:
234         # ["5", "3", "4", "6", "7", "8", "9", "1", "2"]
235         numeros = linea.split()
236
237         # Convertimos cada número de texto a entero.
238         numeros = list(map(int, numeros))
239
240         # Añadimos la fila al Sudoku.
241         miArrayBi.append(numeros)
242

```

```

# COMPROBAR EL SUDOKU

# Llamamos a nuestra función.
resultado = esSudokuCorrecto(miArrayBi)

# MOSTRAR EL RESULTADO

# Si la función devuelve True
if resultado:

    # El Sudoku es correcto.
    print("Correcto")

else:

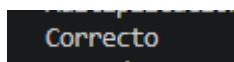
    # El Sudoku es incorrecto.
    print("Incorrecto")

```

Sudoku.in

```
UD00Act2AngelEsteban.py  Profesor.py  Alumno.py  Escola.py  Sudoku.in X
Sudoku.in
1  5 3 4 6 7 8 9 1 2
2  6 7 2 1 9 5 3 4 8
3  1 9 8 3 4 2 5 6 7
4  8 5 9 7 6 1 4 2 3
5  4 2 6 8 5 3 7 9 1
6  7 1 3 9 2 4 8 5 6
7  9 6 1 5 3 7 2 8 4
8  2 8 7 4 1 9 6 3 5
9  3 4 5 2 8 6 1 7 9
```

Resultado:



El sudoku está hecho a propósito para que dé Correcto, si estuviera mal, daría incorrecto.

Actividad 03

```
264
265  # Actividad 03
266
267  # FUNCIÓN PARA CONTAR LOS PATRONES
268
269  def numeroPatrones(text):
270      """
271      Esta función recibe una cadena de texto y devuelve
272      el número total de patrones encontrados.
273
274      Los patrones que tenemos que buscar son:
275          "00"
276          "101"
277          "ABC"
278          "HO"
279
280      Los patrones pueden aparecer solapados.
281      """
282
283      # Guardamos los patrones que tenemos que buscar
284      # en una lista.
285      patrones = ["00", "101", "ABC", "HO"]
286
287      # Esta variable almacenará el número total
288      # de patrones encontrados.
289      contador = 0
```

```

# RECORRER TODOS LOS PATRONES

# Recorremos uno a uno los patrones de la lista.
for patron in patrones:

    # Guardamos la longitud del patrón.

    # Por ejemplo:
    # "00" -> 2
    # "101" -> 3
    # "ABC" -> 3
    # "HO" -> 2
    longitud = len(patron)

    # BUSCAR EL PATRÓN EN EL TEXTO

    # Recorremos todas las posiciones posibles
    # del texto.

    # Utilizamos len(text) - longitud + 1 porque
    # necesitamos asegurarnos de que quede espacio
    # suficiente para que quepa el patrón.
    for i in range(len(text) - longitud + 1):

        # Extraemos del texto una parte que tenga
        # exactamente la misma longitud que el patrón.

        # Por ejemplo, si buscamos "00" y el texto es
        # "000", comprobamos:

```

```

    # posición 0 -> "00"
    # posición 1 -> "00"

    # Por eso encontramos 2 coincidencias.
    trozo = text[i:i + longitud]

    # Comprobamos si el trozo es igual al patrón.
    if trozo == patron:

        # Si coincide, aumentamos el contador
        # en 1.
        contador += 1

    # Cuando terminamos de comprobar todos los patrones,
    # devolvemos el número total de coincidencias.
    return contador

# PEDIR UNA CADENA AL USUARIO

# input() permite al usuario introducir una cadena
# de texto por teclado.
texto = input("Introduce una cadena de texto: ")

```

```

# LLAMAR A LA FUNCIÓN

# Llamamos a numeroPatrones() y le pasamos la cadena
# introducida por el usuario.
resultado = numeroPatrones(texto)

# MOSTRAR EL RESULTADO

# Mostramos por pantalla el número de patrones encontrados.
print("Número de patrones:", resultado)

```

Resultado:

```

Introduce una cadena de texto: 000101ABCABC00HO
Número de patrones: 7

```

Actividad 04

```

360 # Actividad 04
361
362 # FUNCIÓN QUE CUENTA LAS MINAS ADYACENTES
363
364 def contandoMinas(miCampo):
365
366     # Obtenemos el número de filas del campo.
367     filas = len(miCampo)
368
369     # Creamos una matriz nueva para guardar el resultado.
370     # Al principio tendrá todos los valores a 0.
371     resultado = []
372
373     for i in range(filas):
374
375         # Creamos una fila con tantos ceros como columnas
376         # tenga la fila original.
377         fila = [0] * len(miCampo[i])
378
379         # Añadimos la fila a la matriz resultado.
380         resultado.append(fila)
381
382     # RECORRER TODAS LAS POSICIONES DEL CAMPO
383
384     for fila in range(filas):
385
386         # Número de columnas de la fila actual.
387         columnas = len(miCampo[fila])
388

```

```

# RECORRER TODAS LAS POSICIONES DEL CAMPO

for fila in range(filas):

    # Número de columnas de la fila actual.
    columnas = len(miCampo[fila])

    for columna in range(columnas):

        # Si la posición actual es -1, significa que
        # hay una mina.

        # En ese caso dejamos -1 en el resultado.
        if miCampo[fila][columna] == -1:

            resultado[fila][columna] = -1

        else:

            # Contador de minas que hay alrededor
            # de la posición actual.
            contador = 0

            # COMPROBAR LAS 8 POSICIONES ADYACENTES

            # Recorremos las filas que están alrededor.
            for df in range(-1, 2):

                # Recorremos las columnas que están
                # alrededor.
                for dc in range(-1, 2):

```

```

410
411         # Recorremos las columnas que están
412         # alrededor.
413         for dc in range(-1, 2):
414
415             # Calculamos la posición del vecino.
416             nuevaFila = fila + df
417             nuevaColumna = columna + dc
418
419
420             # Comprobamos que la posición esté
421             # dentro de los límites del campo.
422             if (0 <= nuevaFila < filas and
423                 0 <= nuevaColumna < len(miCampo[nuevaFila])):
424
425                 # Comprobamos que no estemos
426                 # comprobando la propia casilla.
427                 if not (df == 0 and dc == 0):
428
429                     # Si encontramos una mina,
430                     # aumentamos el contador.
431
432                     # Según el ejemplo del ejercicio,
433                     # tanto -1 como 1 representan
434                     # una mina.
435                     if (miCampo[nuevaFila][nuevaColumna] == -1 or
436                         miCampo[nuevaFila][nuevaColumna] == 1):
437
438                         contador += 1
439
440

```

```

440
441         # Guardamos el número de minas encontradas
442         # alrededor de la posición.
443         resultado[fila][columna] = contador
444
445
446     # Devolvemos la matriz con el resultado.
447     return resultado
448
449
450 # LEER EL CAMPO POR TECLADO
451
452 # Pedimos al usuario el número de filas.
453 numeroFilas = int(input("Número de filas: "))
454
455
456 # Creamos una lista vacía para guardar el campo.
457 miCampo = []
458
459
460 # Leemos cada fila.
461 for i in range(numeroFilas):
462
463     # Pedimos una fila de números.
464     linea = input("Introduce la fila: ")
465
466     # Separamos los números utilizando los espacios.
467     numeros = linea.split()
468
469     # Convertimos los números de texto a números enteros.
470     numeros = list(map(int, numeros))

```

```

468
469     # Convertimos los números de texto a números enteros.
470     numeros = list(map(int, numeros))
471
472     # Guardamos la fila en el campo.
473     miCampo.append(numeros)
474
475
476 # CALCULAR EL RESULTADO
477
478 # Llamamos a la función contandoMinas().
479 resultado = contandoMinas(miCampo)
480
481
482 # MOSTRAR EL RESULTADO
483
484 print("Resultado:")
485
486 # Recorremos todas las filas del resultado.
487 for fila in resultado:
488
489     # Convertimos los números a texto y los juntamos
490     # utilizando espacios.
491     print(" ".join(map(str, fila)))
492

```

Resultado:

```
Número de filas: 2
Introduce la fila: 0 0 -1 0
Introduce la fila: 0 -1 1 0
Resultado:
1 3 -1 2
1 -1 2 2
```

Actividad 05

```
1  # CLASE PROFESOR
2
3  class Profesor:
4
5      # Constructor de la clase.
6      # Se ejecuta cuando creamos un profesor.
7      def __init__(self, nombre, tipo):
8
9          # Guardamos el nombre del profesor.
10         self.nombre = nombre
11
12         # Guardamos el tipo de profesor.
13         # Puede ser:
14         # ciencias
15         # letras
16         # mixto
17         self.tipo = tipo
18
19
20     # MÉTODO PARA MOSTRAR EL PROFESOR
21
22     def mostrar(self):
23
24         # Devolvemos un texto con la información
25         # del profesor.
26         return self.nombre + " - " + self.tipo
```

```
# CLASE ALUMNO
```

```
class Alumno:
```

```
    # Constructor de la clase.
```

```
    def __init__(self, nombre, curso, profesor):
```

```
        # Guardamos el nombre del alumno.
```

```
        self.nombre = nombre
```

```
        # Guardamos el curso.
```

```
        self.curso = curso
```

```
        # Guardamos el profesor responsable.
```

```
        self.profesor = profesor
```

```
    # MÉTODO PARA MOSTRAR EL ALUMNO
```

```
    def mostrar(self):
```

```
        # Devolvemos la información del alumno.
```

```
        return (
```

```
            self.nombre
```

```
            + " - Curso: "
```

```
            + self.curso
```

```
            + " - Profesor: "
```

```
            + self.profesor.nombre
```

```
        )
```

```
# CLASE ESCOLA
```

```
class Escola:
```

```
    # Constructor de la escuela.
```

```
    def __init__(self, nombre, localidad, responsable):
```

```
        # Información básica de la escuela.
```

```
        self.nombre = nombre
```

```
        self.localidad = localidad
```

```
        self.responsable = responsable
```

```
        # Diccionario donde guardaremos los profesores.
```

```
        # La clave será el nombre del profesor.
```

```
        self.profesores = {}
```

```
        # Diccionario donde guardaremos los grupos.
```

```
        # Cada grupo tendrá una lista de alumnos.
```

```
        self.grupos = {}
```

```

# PROFESORES

# AÑADIR PROFESOR

def añadirProfesor(self, profesor):

    # Comprobamos si el profesor ya existe.
    if profesor.nombre in self.profesores:

        print("El profesor ya existe.")

    else:

        # Guardamos el profesor en el diccionario.
        self.profesores[profesor.nombre] = profesor

        print("Profesor añadido correctamente.")

# ELIMINAR PROFESOR

def eliminarProfesor(self, nombre):

    # Comprobamos si el profesor existe.
    if nombre in self.profesores:

        # Lo eliminamos.
        del self.profesores[nombre]

        print("Profesor eliminado.")

```

```

    else:

        print("El profesor no existe.")

# MOSTRAR PROFESORES

def mostrarProfesores(self):

    print("\n--- PROFESORES ---")

    # Recorremos todos los profesores.
    for profesor in self.profesores.values():

        print(profesor.mostrar())

```

```
# GRUPOS

# AÑADIR GRUPO

def añadirGrupo(self, nombreGrupo):

    # Comprobamos si el grupo ya existe.
    if nombreGrupo in self.grupos:

        print("El grupo ya existe.")

    else:

        # Creamos una lista vacía para los alumnos.
        self.grupos[nombreGrupo] = []

        print("Grupo añadido correctamente.")

# AÑADIR ALUMNO A UN GRUPO

def añadirAlumno(self, nombreGrupo, alumno):

    # Comprobamos que el grupo exista.
    if nombreGrupo not in self.grupos:

        print("El grupo no existe.")
        return
```

```

# Comprobamos si el alumno ya está en algún grupo.
for grupo in self.grupos.values():

    for alumnoExistente in grupo:

        if alumnoExistente.nombre == alumno.nombre:

            print("El alumno ya está en la escuela.")
            return

# Añadimos el alumno al grupo.
self.grupos[nombreGrupo].append(alumno)

print("Alumno añadido correctamente.")

# ELIMINAR ALUMNO

def eliminarAlumno(self, nombreGrupo, nombreAlumno):

    # Comprobamos que el grupo exista.
    if nombreGrupo not in self.grupos:

        print("El grupo no existe.")
        return

```

```

# Recorremos los alumnos del grupo.
for alumno in self.grupos[nombreGrupo]:

    # Buscamos el alumno por su nombre.
    if alumno.nombre == nombreAlumno:

        # Lo eliminamos.
        self.grupos[nombreGrupo].remove(alumno)

        print("Alumno eliminado.")
        return

print("El alumno no existe.")

# MOSTRAR ALUMNOS

def mostrarAlumnos(self):

    print("\n--- ALUMNOS ---")

    # Recorremos todos los grupos.
    for nombreGrupo, alumnos in self.grupos.items():

        print("\nGrupo:", nombreGrupo)

```

```

        print("\nGrupo:", nombreGrupo)

        # Mostramos los alumnos del grupo.
        for alumno in alumnos:
            print(alumno.mostrar())

# MOSTRAR INFORMACIÓN DE LA ESCUELA

def mostrarInformacion(self):
    print("ESCUELA:", self.nombre)
    print("LOCALIDAD:", self.localidad)
    print("RESPONSABLE:", self.responsable)

    self.mostrarProfesores()

    self.mostrarAlumnos()

```

```

# Actividad 05

# IMPORTAMOS LAS CLASES

from Profesor import Profesor
from Alumno import Alumno
from Escuela import Escuela

# CREAR PROFESORES

# Creamos el primer profesor.
profesor1 = Profesor(
    "Juan",
    "ciencias"
)

# Creamos el segundo profesor.
profesor2 = Profesor(
    "María",
    "letras"
)

# Creamos el tercer profesor.
profesor3 = Profesor(
    "Pedro",
    "mixto"
)

```

```
# CREAR LA ESCUELA

escuela = Escola(
    "IES Mediterráneo",
    "Valencia",
    "Laura"
)

# AÑADIR PROFESORES A LA ESCUELA

escuela.añadirProfesor(profesor1)

escuela.añadirProfesor(profesor2)

escuela.añadirProfesor(profesor3)

# CREAR GRUPOS

escuela.añadirGrupo("1ESO-A")

escuela.añadirGrupo("2ESO-B")

# CREAR ALUMNOS

alumno1 = Alumno(
    "Carlos",
    "1ESO",
    profesor1
)
```

```

558     alumno2 = Alumno(
559         "Ana",
560         "1ESO",
561         profesor2
562     )
563
564     alumno3 = Alumno(
565         "Luis",
566         "2ESO",
567         profesor3
568     )
569
570
571     # AÑADIR ALUMNOS A LOS GRUPOS
572
573     escuela.añadirAlumno(
574         "1ESO-A",
575         alumno1
576     )
577
578     escuela.añadirAlumno(
579         "1ESO-A",
580         alumno2
581     )
582
583     escuela.añadirAlumno(
584         "2ESO-B",
585         alumno3
586     )
587

```

```

# MOSTRAR TODA LA INFORMACIÓN

escuela.mostrarInformacion()

# PROBAR LA ELIMINACIÓN DE UN ALUMNO

print("\nEliminando a Carlos...")

escuela.eliminarAlumno(
    "1ESO-A",
    "Carlos"
)

# MOSTRAR DE NUEVO LOS ALUMNOS

escuela.mostrarAlumnos()

```

Resultado:

```
Profesor añadido correctamente.  
Profesor añadido correctamente.  
Profesor añadido correctamente.  
Grupo añadido correctamente.  
Grupo añadido correctamente.  
Alumno añadido correctamente.  
Alumno añadido correctamente.  
Alumno añadido correctamente.  
ESCUELA: IES Mediterráneo  
LOCALIDAD: Valencia  
RESPONSABLE: Laura
```

```
--- PROFESORES ---  
Juan - ciencias  
María - letras  
Pedro - mixto
```

```
--- ALUMNOS ---  
  
Grupo: 1ESO-A  
Carlos - Curso: 1ESO - Profesor: Juan  
Ana - Curso: 1ESO - Profesor: María  
  
Grupo: 2ESO-B  
Luis - Curso: 2ESO - Profesor: Pedro  
  
Eliminando a Carlos...  
Alumno eliminado.
```

```
--- ALUMNOS ---  
  
Grupo: 1ESO-A  
Ana - Curso: 1ESO - Profesor: María  
  
Grupo: 2ESO-B  
Luis - Curso: 2ESO - Profesor: Pedro
```

Actividad 06

Alumno 2

```
Alumno2.py > ...
1  # CLASE ALUMNO - ACTIVIDAD 06
2
3  # Importamos las herramientas de SQLAlchemy.
4  from sqlalchemy import Column, Integer, String, ForeignKey
5
6  # relationship() permite relacionar unas clases
7  # con otras.
8  from sqlalchemy.orm import relationship
9
10 # Importamos Base y Escola2.
11 from Escola2 import Base, Escola2
12
13 # Importamos Profesor2.
14 from Profesor2 import Profesor2
15
16
17 # CLASE ALUMNO2
18
19 class Alumno2(Base):
20
21     # Nombre de la tabla en SQLite.
22     __tablename__ = "alumnos2"
23
24     # Identificador único del alumno.
25     id = Column(Integer, primary_key=True)
26
27     # Nombre del alumno.
28     nombre = Column(String, nullable=False)
29
30     # Curso del alumno.
31     curso = Column(String, nullable=False)
32
33     # Grupo al que pertenece.
34     grupo = Column(String, nullable=False)
```

```

37     # RELACIÓN CON ESCOLA2
38
39     # ID de la escuela a la que pertenece el alumno.
40     escola_id = Column(
41         Integer,
42         ForeignKey("escoles2.id"),
43         nullable=False
44     )
45
46     # Relación entre Alumno2 y Escola2.
47     escola = relationship("Escola2")
48
49
50     # RELACIÓN CON PROFESOR2
51
52     # ID del profesor responsable del alumno.
53     profesor_id = Column(
54         Integer,
55         ForeignKey("profesores2.id"),
56         nullable=False
57     )
58
59     # Relación entre Alumno2 y Profesor2.
60     profesor = relationship("Profesor2")
61
62

```

```
# Relación entre Alumno2 y Profesor2.
profesor = relationship("Profesor2")

# CONSTRUCTOR

def __init__(
    self,
    nombre,
    curso,
    grupo,
    escola,
    profesor
):

    self.nombre = nombre
    self.curso = curso
    self.grupo = grupo

    # Guardamos la escuela relacionada.
    self.escola = escola

    # Guardamos el profesor responsable.
    self.profesor = profesor
```

```
# MOSTRAR INFORMACIÓN

def __str__(self):

    return (
        f"Alumno: {self.nombre} | "
        f"Curso: {self.curso} | "
        f"Grupo: {self.grupo} | "
        f"Profesor: {self.profesor.nombre}"
    )
```

Profesor 2

```
Profesor2.py > ...
1  # CLASE PROFESOR - ACTIVIDAD 06
2
3  # Importamos las herramientas de SQLAlchemy.
4  from sqlalchemy import Column, Integer, String, ForeignKey
5
6  # relationship() permite establecer relaciones
7  # entre nuestras clases.
8  from sqlalchemy.orm import relationship
9
10 # Importamos Base y Escola2 desde Escola2.py.
11 from Escola2 import Base, Escola2
12
13
14 # CLASE PROFESOR2
15
16 class Profesor2(Base):
17
18     # Nombre de la tabla en SQLite.
19     __tablename__ = "profesores2"
20
21     # Identificador único del profesor.
22     id = Column(Integer, primary_key=True)
23
24     # Nombre del profesor.
25     nombre = Column(String, nullable=False)
26
27     # Tipo de profesor.
28     # Puede ser ciencias, letras o mixto.
29     tipo = Column(String, nullable=False)
30
31
```

```

32     # RELACIÓN CON ESCOLA2
33
34     # Guardamos el ID de la escuela a la que pertenece
35     # el profesor.
36     escola_id = Column(
37         Integer,
38         ForeignKey("escoles2.id"),
39         nullable=False
40     )
41
42     # Creamos la relación entre Profesor2 y Escola2.
43     escola = relationship("Escola2")
44
45
46     # CONSTRUCTOR
47
48     def __init__(self, nombre, tipo, escola):
49
50         self.nombre = nombre
51         self.tipo = tipo
52
53         # Guardamos la escuela relacionada.
54         self.escola = escola
55

```

```

# MOSTRAR INFORMACIÓN

def __str__(self):

    return (
        f"Profesor: {self.nombre} | "
        f"Tipo: {self.tipo}"
    )

```

Escola 2

```
Escola2.py > ...
1  # CLASE ESCOLA - ACTIVIDAD 06
2
3  # Importamos las herramientas de SQLAlchemy que necesitamos
4  # para crear las columnas de nuestra tabla.
5  from sqlalchemy import Column, Integer, String
6
7  # Importamos declarative_base().
8  # Nos permite crear clases que SQLAlchemy puede convertir
9  # en tablas de una base de datos.
10 from sqlalchemy.orm import declarative_base
11
12
13 # BASE DE SQLALCHEMY
14
15 # Creamos la clase Base.
16
17 # Las clases que queramos guardar en la base de datos
18 # heredarán de esta clase.
19 Base = declarative_base()
20
21
22 # CLASE ESCOLA2
23
24 class Escola2(Base):
25
26     # Nombre de la tabla en SQLite.
27     __tablename__ = "escoles2"
28
29     # Identificador único de la escuela.
30     # primary_key=True significa que es la clave primaria.
31     id = Column(Integer, primary_key=True)
32
33     # Nombre de la escuela.
34     nombre = Column(String, nullable=False)
```

```

# Localidad de la escuela.
localidad = Column(String, nullable=False)

# Nombre del responsable de la escuela.
responsable = Column(String, nullable=False)

# CONSTRUCTOR

def __init__(self, nombre, localidad, responsable):
    self.nombre = nombre
    self.localidad = localidad
    self.responsable = responsable

# MOSTRAR INFORMACIÓN

def __str__(self):
    return (
        f"Escuela: {self.nombre} | "
        f"Localidad: {self.localidad} | "
        f"Responsable: {self.responsable}"
    )

```

Código main

```
# Actividad 06

# Importamos create_engine().
# Sirve para conectar nuestro programa con SQLite.
from sqlalchemy import create_engine

# sessionmaker() permite crear sesiones
# para trabajar con la base de datos.
from sqlalchemy.orm import sessionmaker

# IMPORTAR NUESTRAS CLASES

from Escola2 import Base, Escola2
from Profesor2 import Profesor2
from Alumno2 import Alumno2

# CREAR LA BASE DE DATOS

# Creamos una base de datos SQLite llamada:

# escuela2.db

# Si el archivo no existe, SQLite lo creará.
engine = create_engine("sqlite:///escuela2.db")
```

```

637
638 # CREAR LAS TABLAS
639
640 # SQLAlchemy utiliza nuestras clases para crear
641 # las tablas correspondientes en la base de datos.
642 Base.metadata.create_all(engine)
643
644
645 # CREAR UNA SESIÓN
646
647 # Creamos una clase para poder trabajar
648 # con la base de datos.
649 Session = sessionmaker(bind=engine)
650
651 # Creamos una sesión.
652 session = Session()
653
654
655 # CREAR UNA ESCUELA
656
657 escuela = Escola2(
658     "IES Mediterráneo",
659     "Valencia",
660     "Laura"
661 )
662
663
664 # Añadimos la escuela a la sesión.
665 session.add(escuela)
666
667 # Guardamos los cambios.
668 session.commit()

```

```
670
671 # CREAR PROFESORES
672
673 profesor1 = Profesor2(
674     "Juan",
675     "ciencias",
676     escuela
677 )
678
679 profesor2 = Profesor2(
680     "María",
681     "letras",
682     escuela
683 )
684
685
686 # Añadimos los profesores.
687 session.add(profesor1)
688 session.add(profesor2)
689
690 # Guardamos los cambios.
691 session.commit()
692
693
694 # CREAR ALUMNOS
695
696 alumno1 = Alumno2(
697     "Carlos",
698     "1ESO",
699     "A",
700     escuela,
701     profesor1
702 )
```

```

703
704     alumno2 = Alumno2(
705         "Ana",
706         "1ESO",
707         "A",
708         escuela,
709         profesor2
710     )
711
712     alumno3 = Alumno2(
713         "Luis",
714         "2ESO",
715         "B",
716         escuela,
717         profesor1
718     )
719
720
721     # GUARDAR LOS ALUMNOS
722
723     # Añadimos los tres alumnos a la sesión.
724     session.add(alumno1)
725     session.add(alumno2)
726     session.add(alumno3)
727
728     # Confirmamos los cambios.
729
730     # En este momento los alumnos quedan guardados
731     # físicamente en la base de datos.
732     session.commit()
733

```

```

734
735     print("Alumnos guardados correctamente.")
736
737
738     # RECUPERAR LOS ALUMNOS
739
740     # Consultamos todos los alumnos almacenados
741     # en la tabla alumnos2.
742     alumnos = session.query(Alumno2).all()
743
744
745     # MOSTRAR LOS ALUMNOS
746
747     print("\n--- ALUMNOS GUARDADOS EN LA BASE DE DATOS ---")
748
749     # Recorremos la lista de alumnos.
750     for alumno in alumnos:
751         print(alumno)
752
753
754
755     # CERRAR LA SESIÓN
756
757     session.close()
758
759     print("\nPrograma terminado.")
760

```

Resultado:

```

--- ALUMNOS GUARDADOS EN LA BASE DE DATOS ---
Alumno: Carlos | Curso: 1ESO | Grupo: A | Profesor: Juan
Alumno: Ana | Curso: 1ESO | Grupo: A | Profesor: María
Alumno: Luis | Curso: 2ESO | Grupo: B | Profesor: Juan

```

Actividad 07

Para este ejercicio necesitamos ejecutarlo poniendo en consola lo siguiente para que reciba los ficheros que necesita:

“python UD00Act2AngelEsteban.py ejemploEntrada.txt
ejemploSalida.txt”

```
# Actividad 07

# Programa que:
# - Lee números desde un fichero de entrada.
# - Comprueba cuáles son palíndromos.
# - Comprueba cuáles son primos.
# - Escribe los resultados en un fichero de salida.

# Además, las funciones esPalindromo() y esPrimo()
# tienen pruebas unitarias mediante doctest.

# IMPORTACIONES

# sys nos permite acceder a los parámetros
# que escribimos al ejecutar el programa.
import sys

# math nos permite utilizar sqrt(), es decir,
# la raíz cuadrada de un número.
import math

# doctest permite ejecutar los ejemplos que tenemos
# escritos dentro de las funciones como pruebas.
import doctest
```

```
def esPalindromo(numero):
    """
    Comprueba si un número es palíndromo.

    Un número palíndromo es aquel que se lee igual
    de izquierda a derecha que de derecha a izquierda.

    Ejemplos:

    >>> esPalindromo(5)
    True

    >>> esPalindromo(11)
    True

    >>> esPalindromo(22)
    True

    >>> esPalindromo(13)
    False

    >>> esPalindromo(123)
    False
    """

    # Convertimos el número a texto.
    texto = str(numero)
```

```
819
820     # [::-1] sirve para invertir una cadena.
821     texto_invertido = texto[::-1]
822
823     # Comparamos el número original con el invertido.
824     return texto == texto_invertido
825
```

```
def esPrimo(numero):  
    """  
    Comprueba si un número es primo.  
  
    Un número primo solamente puede dividirse  
    exactamente entre 1 y entre él mismo.  
  
    Ejemplos:  
  
    >>> esPrimo(2)  
    True  
  
    >>> esPrimo(5)  
    True  
  
    >>> esPrimo(11)  
    True  
  
    >>> esPrimo(13)  
    True  
  
    >>> esPrimo(4)  
    False  
  
    >>> esPrimo(15)  
    False  
  
    >>> esPrimo(1)  
    False  
    """
```

```
# Los números menores que 2 no son primos.
if numero < 2:
    return False

# El 2 es primo.
if numero == 2:
    return True

# Los números pares mayores que 2 no son primos.
if numero % 2 == 0:
    return False

# Comprobamos posibles divisores desde 3
# hasta la raíz cuadrada del número.
limite = int(math.sqrt(numero)) + 1

for divisor in range(3, limite, 2):
    # Si la división es exacta,
    # el número no es primo.
    if numero % divisor == 0:
        return False

# Si no hemos encontrado ningún divisor,
# el número es primo.
return True
```

```

def main():
    """
    Función principal del programa.

    Recibe dos parámetros desde la consola:

        1. Fichero de entrada.
        2. Fichero de salida.
    """

    # COMPROBAR LOS PARÁMETROS

    # sys.argv contiene los parámetros escritos
    # al ejecutar el programa.

    # sys.argv[0] -> nombre del programa
    # sys.argv[1] -> fichero de entrada
    # sys.argv[2] -> fichero de salida

    # Por tanto necesitamos 3 elementos.
    if len(sys.argv) != 3:

        print(
            "Uso: python UD00Act2AngelEsteban.py "
            "ficheroEntrada.txt ficheroSalida.txt"
        )

    return

```

```

# GUARDAR LOS NOMBRES DE LOS FICHEROS

# Guardamos el nombre del fichero de entrada.
fichero_entrada = sys.argv[1]

# Guardamos el nombre del fichero de salida.
fichero_salida = sys.argv[2]

# LEER EL FICHERO DE ENTRADA

# Creamos una lista vacía donde guardaremos
# todos los números.
numeros = []

# Abrimos el fichero de entrada en modo lectura.
with open(fichero_entrada, "r", encoding="utf-8") as archivo:

    # Recorremos todas las líneas del fichero.
    for linea in archivo:

        # Quitamos espacios y salto de línea.
        linea = linea.strip()

        # Si la línea no está vacía
        if linea != "":

            # Convertimos el texto a número entero.
            numero = int(linea)

            # Añadimos el número a la lista.
            numeros.append(numero)

```

```
# CONTADORES

# Contador de números palíndromos.
cantidad_palindromos = 0

# Contador de números primos.
cantidad_primos = 0

# Lista para guardar los números que son
# simultáneamente primos y palíndromos.
primos_y_palindromos = []

# COMPROBAR LOS NÚMEROS

# Recorremos todos los números leídos.
for numero in numeros:

    # Comprobamos si es palíndromo.
    palindromo = esPalindromo(numero)

    # Comprobamos si es primo.
    primo = esPrimo(numero)

    # Si es palíndromo
    if palindromo:

        # Aumentamos el contador.
        cantidad_palindromos += 1
```

```

# Si es primo
if primo:

    # Aumentamos el contador.
    cantidad_primos += 1

# Si es primo Y palíndromo
if primo and palindromo:

    # Lo añadimos a la lista.
    primos_y_palindromos.append(numero)

# CREAR EL FICHERO DE SALIDA

# Abrimos el fichero de salida en modo escritura.
with open(fichero_salida, "w", encoding="utf-8") as archivo:

    # Primera línea:
    # cantidad de números palíndromos.
    archivo.write(
        f"Hay {cantidad_palindromos} números palíndromos.\n"
    )

    # Segunda línea:
    # cantidad de números primos.
    archivo.write(
        f"Hay {cantidad_primos} números primos.\n"
    )

```

```

# NÚMEROS QUE SON PRIMOS Y PALÍNDROMOS

# Recorremos la lista.
for numero in primos_y_palindromos:

    # Escribimos un número por línea.
    archivo.write(f"{numero}\n")

# EJECUCIÓN DEL PROGRAMA

if __name__ == "__main__":

    # Si ejecutamos:
    # python UD00Act2AngelEsteban.py entrada.txt salida.txt
    # se ejecutará el programa.

    main()

```

El resultado de este ejercicio no se muestra en consola, si no en los .txt, estos son los de entrada y salida:

```

ejemploEntrada.txt
1    5
2    7
3   11
4   13
5   14
6   15
7   22

```

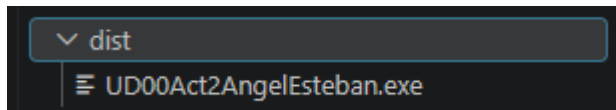
```

ejemploSalida.txt
1   Hay 4 números palíndromos.
2   Hay 4 números primos.
3    5
4    7
5   11
6

```

Para hacer el .exe ejecutaríamos este comando y nos haría una carpeta con él:

“python -m PyInstaller --onefile UD00Act2AngelEsteban.py”



Nombre	Fecha de modificación	Tipo	Tamaño
 UD00Act2AngelEsteban.exe	04/10/2026 17:30	Aplicación	14.869 KB

Y para probarlo haríamos este comando, pero daría el mismo resultado que hemos visto:

“.\dist\UD00Act2AngelEsteban.exe ejemploEntrada.txt
ejemploSalida.txt”

Tests Actividad 07

A continuación, los tests de la actividad:

```
PS C:\Users\angel\Desktop\SGEntregar> python -m doctest -v UD00Act2AngelEsteban.py
```

```
Trying:
    esPalindromo(5)
Expecting:
    True
ok
Trying:
    esPalindromo(11)
Expecting:
    True
ok
Trying:
    esPalindromo(22)
```

```
Expecting:
    True
ok
Trying:
    esPalindromo(13)
Expecting:
    False
ok
Trying:
    esPalindromo(123)
Expecting:
    False
```

```
ok
Trying:
  esPrimo(2)
Expecting:
  True
ok
Trying:
  esPrimo(5)
Expecting:
  True
ok
Trying:
```

```
  esPrimo(11)
Expecting:
  True
ok
Trying:
  esPrimo(13)
Expecting:
  True
ok
Trying:
  esPrimo(4)
Expecting:
```

```
Expecting:
  False
ok
Trying:
  esPrimo(15)
Expecting:
  False
ok
Trying:
  esPrimo(1)
Expecting:
  False
```

```
  esPrimo(1)
Expecting:
  False
ok
```

```
2 items passed all tests:
  5 tests in UD00Act2AngelEsteban.esPalindromo
  7 tests in UD00Act2AngelEsteban.esPrimo
12 tests in 10 items.
12 passed.
Test passed.
```

Actividad 08

```
1  # Actividad 08
2
3  # ORGANIZADOR DE ARCHIVOS
4
5  # Importamos os.
6  # Nos permite trabajar con carpetas y archivos.
7  import os
8
9  # Importamos shutil.
10 # Nos permite mover archivos de una carpeta a otra.
11 import shutil
12
13
14 # FUNCIÓN PARA ORGANIZAR LOS ARCHIVOS
15
16 def organizarArchivos(extensiones):
17
18     # Obtenemos la carpeta actual.
19
20     # "." significa la carpeta en la que estamos trabajando.
21     carpeta_actual = "."
22
23
24     # RECORRER LAS EXTENSIONES
25
26     # Recorreremos una por una las extensiones de la lista.
27
28     # Por ejemplo:
29
30     # ["png", "mp4", "doc"]
31
```

```
31
32     # Primero extension será "png".
33     # Después será "mp4".
34     # Finalmente será "doc".
35     for extension in extensiones:
36
37         # Creamos el nombre de la carpeta.
38
39         # Si extension = "png"
40         # carpeta = "png"
41         carpeta = extension
42
43
44         # CREAR LA CARPETA
45
46         # Comprobamos si la carpeta todavía no existe.
47         if not os.path.exists(carpeta):
48
49             # Creamos la carpeta.
50             os.mkdir(carpeta)
51
52
53         # RECORRER LOS ARCHIVOS DE LA CARPETA ACTUAL
54
55         # os.listdir() devuelve todos los archivos y carpetas
56         # que hay dentro de la carpeta actual.
57         archivos = os.listdir(carpeta_actual)
58
59
```

```
# Recorremos todos los elementos encontrados.
for archivo in archivos:

    # Creamos la ruta completa del archivo.
    ruta_archivo = os.path.join(
        carpeta_actual,
        archivo
    )

    # COMPROBAR QUE SEA UN ARCHIVO

    # No queremos intentar mover carpetas.
    if os.path.isfile(ruta_archivo):

        # COMPROBAR LA EXTENSIÓN

        # Comprobamos si el archivo termina con
        # la extensión que estamos buscando.

        # Por ejemplo:

        # foto.png

        # termina en ".png".
        if archivo.lower().endswith(
            "." + extension.lower()
        ):

```

```

# CREAR LA RUTA DE DESTINO

destino = os.path.join(
    carpeta,
    archivo
)

# MOVER EL ARCHIVO

shutil.move(
    ruta_archivo,
    destino
)

# LISTA DE EXTENSIONES

# Creamos una lista con las extensiones
# que queremos organizar.
extensiones = [
    "png",
    "mp4",
    "doc"
]

# EJECUTAR EL ORGANIZADOR

organizarArchivos(extensiones)

```

```

# Mostramos un mensaje cuando hemos terminado.
print("Archivos organizados correctamente.")

```

No se ejecuta la prueba para evitar modificar/mover los archivos del equipo, ya que el programa organiza los archivos en carpetas según su extensión.

Actividad 09

```
# Actividad 09

# GENERAR CÓDIGOS DE BARRAS

import sys
import csv
from barcode import EAN13
from barcode.writer import ImageWriter

# Comprobamos que se haya pasado un archivo CSV
# como parámetro al ejecutar el programa.
if len(sys.argv) != 2:

    # Si no se ha pasado exactamente un archivo,
    # mostramos cómo utilizar el programa.
    print("Uso: python UD00Act2AngelEsteban.py alumnos.csv")

else:

    # Guardamos el nombre del archivo CSV.
    fichero_csv = sys.argv[1]

    # Abrimos el archivo CSV.
    with open(fichero_csv, "r", encoding="utf-8", newline="") as archivo:

        # Creamos el lector del CSV.
        lector = csv.reader(archivo)

        # Recorremos todos los alumnos del archivo.
        for fila in lector:
```

```

# Si encontramos una línea vacía, la ignoramos.
if not fila:
    continue

# Primera columna: nombre del alumno.
nombre = fila[0].strip()

# Segunda columna: ID del alumno.
identificador = fila[1].strip()

# EAN-13 necesita 12 números.
# Rellenamos el ID con ceros por la izquierda.
codigo = identificador.zfill(12)

# Creamos el código de barras EAN-13.
codigo_barras = EAN13(
    codigo,
    writer=ImageWriter()
)

# Guardamos el código de barras.
# Se creará automáticamente como PNG.
# Por ejemplo: "Alumne 01.png"
codigo_barras.save(nombre)

# Avisamos de que hemos terminado.
print("Códigos de barras generados correctamente.")

```

```

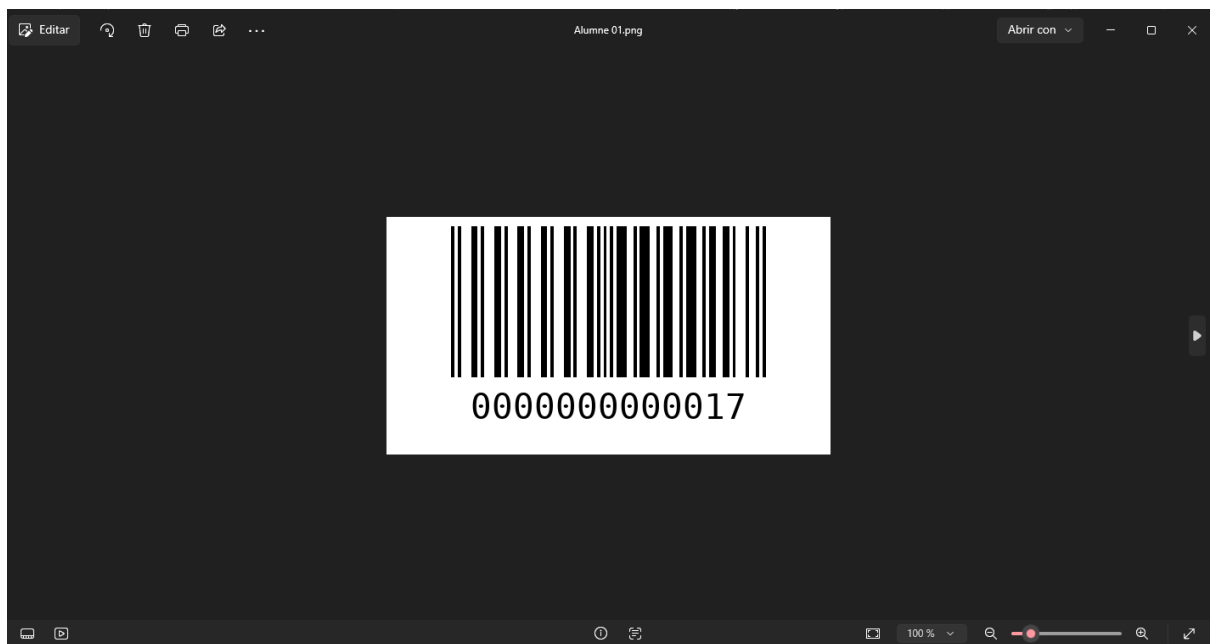
alumnos.csv
1  "Alumne 01","1"
2  "Alumne 02","2"

```

Resultado:

```
Códigos de barras generados correctamente.
```

 Alumne 01.png	05/10/2026 22:25	Archivo PNG	3 KB
 Alumne 02.png	05/10/2026 22:25	Archivo PNG	3 KB



Actividad 10

```
# Actividad 10

# Importamos os para poder trabajar con carpetas y archivos.
import os

# Importamos cv2 para poder leer las imágenes PNG.
import cv2

# Importamos decode de pyzbar para detectar y leer
# los códigos de barras que hay dentro de las imágenes.
from pyzbar.pyzbar import decode

# CARPETA DONDE ESTÁN LOS PNG

# "." significa la carpeta actual.
# Es decir, la misma carpeta donde ejecutamos el programa.
carpeta = "."

# RECORREMOS TODOS LOS ARCHIVOS DE LA CARPETA

# os.listdir() nos devuelve todos los archivos y carpetas
# que existen dentro de la carpeta indicada.
for archivo in os.listdir(carpeta):

    # Comprobamos si el archivo termina en ".png".
    if archivo.lower().endswith(".png"):

        # Creamos la ruta completa del archivo.
        ruta = os.path.join(carpeta, archivo)
```

```

# Leemos la imagen PNG utilizando OpenCV.
imagen = cv2.imread(ruta)

# Intentamos detectar los códigos de barras
# que hay dentro de la imagen.
codigos = decode(imagen)

# Comprobamos si no se ha encontrado ningún código.
if not codigos:

    print("No se ha encontrado ningún código en:", archivo)

else:

    # Recorremos los códigos encontrados.
    for codigo_barras in codigos:

        # Extraemos los datos que contiene el código.
        # barcode.data devuelve los datos como bytes.
        datos = codigo_barras.data.decode("utf-8")

        # El nombre del alumno es el nombre del PNG
        # sin la extensión ".png".
        nombre_alumno = os.path.splitext(archivo)[0]

        # Mostramos el nombre del alumno y su ID.
        print("Alumno:", nombre_alumno, "- ID:", datos)

```

Resultado:

```

Alumno: Alumne 01 - ID: 0000000000017
Alumno: Alumne 02 - ID: 0000000000024

```

Actividad 11

Para esta actividad necesitamos ejecutarlo con una versión de python anterior porque Kivy no es compatible con las más nuevas.

```
PS C:\Users\angel\Desktop\SGEntregar> py -3.12 UD00Act2.11AngelEsteban.py
```

```
1  # ACTIVIDAD 11 - INTERFAZ GRÁFICA CON KIVY
2
3  # Importamos hashlib para poder calcular el SHA1
4  # de la contraseña que escribe el usuario.
5  import hashlib
6
7  # Importamos csv para poder leer el archivo users.csv.
8  import csv
9
10 # Importamos App, que es la clase principal de Kivy
11 # para crear una aplicación.
12 from kivy.app import App
13
14 # BoxLayout permite colocar los elementos de la ventana
15 # uno debajo de otro.
16 from kivy.uix.boxlayout import BoxLayout
17
18 # Label sirve para mostrar textos.
19 from kivy.uix.label import Label
20
21 # TextInput sirve para que el usuario pueda escribir.
22 from kivy.uix.textinput import TextInput
23
24 # Button sirve para crear el botón.
25 from kivy.uix.button import Button
26
```

```

28 # CLASE PRINCIPAL DE LA APLICACIÓN
29
30 class LoginApp(App):
31
32     # build() crea la ventana y todos sus elementos.
33
34     def build(self):
35
36         # Creamos un diccionario donde guardaremos:
37         # usuario -> contraseña SHA1
38         self.usuarios = {}
39
40         # Abrimos el archivo users.csv.
41         with open("users.csv", "r", encoding="utf-8") as archivo:
42
43             # Leemos el archivo usando ":" como separador,
44             # porque el ejercicio tiene este formato:
45             # "usuario":"passwordenSHA1"
46             lector = csv.reader(archivo, delimiter=":")
47
48             # Recorremos todas las líneas del archivo.
49             for fila in lector:
50
51                 # Comprobamos que la línea tenga usuario y contraseña.
52                 if len(fila) >= 2:
53
54                     # Quitamos espacios y comillas del usuario.
55                     usuario = fila[0].strip().strip('\'')
56
57                     # Quitamos espacios y comillas del SHA1.
58                     password_sha1 = fila[1].strip().strip('\'')
59

```

```
        # Guardamos el usuario y su SHA1.
        self.usuarios[usuario] = password_sha1

# Creamos el contenedor principal.
# Los elementos se colocarán verticalmente.
layout = BoxLayout(
    orientation="vertical",
    padding=20,
    spacing=10
)

# Creamos la etiqueta "Usuario".
etiqueta_usuario = Label(
    text="Usuario"
)

# Creamos el campo donde el usuario escribirá
# su nombre de usuario.
self.campo_usuario = TextInput(
    multiline=False
)

# Creamos la etiqueta "Contraseña".
etiqueta_password = Label(
    text="Contraseña"
)
```

```
# Creamos el campo para introducir la contraseña.

# password=True hace que los caracteres aparezcan
# ocultos.
self.campo_password = TextInput(
    multiline=False,
    password=True
)

# Creamos el botón.
boton = Button(
    text="COMPROBAR"
)

# Cuando se pulse el botón, llamaremos al método
# comprobar_usuario().
boton.bind(
    on_press=self.comprobar_usuario
)

# Creamos una etiqueta vacía.
# Aquí mostraremos "OK" o "ERROR".
self.resultado = Label(
    text=""
)

# Añadimos la etiqueta Usuario a la ventana.
layout.add_widget(etiqueta_usuario)
```

```
# Añadimos la etiqueta Usuario a la ventana.
layout.addWidget(etiqueta_usuario)

# Añadimos el campo de usuario.
layout.addWidget(self.campo_usuario)

# Añadimos la etiqueta Contraseña.
layout.addWidget(etiqueta_password)

# Añadimos el campo de contraseña.
layout.addWidget(self.campo_password)

# Añadimos el botón.
layout.addWidget(boton)

# Añadimos la etiqueta donde aparecerá
# OK o ERROR.
layout.addWidget(self.resultado)

# Devolvemos el diseño completo de la ventana.
return layout
```

```
# Este método se ejecuta cuando pulsamos COMPROBAR.

def comprobar_usuario(self, instancia):

    # Obtenemos el usuario escrito en el campo.
    usuario = self.campo_usuario.text

    # Obtenemos la contraseña escrita.
    password = self.campo_password.text

    # Convertimos la contraseña a SHA1.
    password_sha1 = hashlib.sha1(
        password.encode("utf-8")
    ).hexdigest()

    # Comprobamos primero si el usuario existe.
    if usuario in self.usuarios:

        # Obtenemos el SHA1 guardado para ese usuario.
        password_guardada = self.usuarios[usuario]

        # Comparamos el SHA1 escrito con el guardado.
        if password_sha1 == password_guardada:

            # Si coinciden, mostramos OK.
            self.resultado.text = "OK"

        else:

            # Si no coinciden, mostramos ERROR.
            self.resultado.text = "ERROR"
```

```
        else:

            # Si el usuario no existe, mostramos ERROR.
            self.resultado.text = "ERROR"

# INICIAMOS LA APLICACIÓN

# Creamos la aplicación y la ejecutamos.
LoginApp().run()
```

Resultado:



Actividad 12

```
1  # ACTIVIDAD 12 - RECONOCIMIENTO DE TEXTO EN IMÁGENES
2
3  # Importamos OpenCV.
4  # Lo utilizaremos para cargar y manipular la imagen.
5  import cv2
6
7  # Importamos pytesseract.
8  # Esta librería permite utilizar Tesseract OCR desde Python.
9  import pytesseract
10
11
12  # IMAGEN QUE QUEREMOS ANALIZAR
13
14  # Indicamos el nombre de la imagen que queremos leer.
15
16  # La imagen debe estar en la misma carpeta que este programa.
17  nombre_imagen = "imagen.png"
18
19
20  # CARGAMOS LA IMAGEN
21
22  # cv2.imread() carga la imagen desde el archivo.
23  imagen = cv2.imread(nombre_imagen)
24
25
26  # Comprobamos que la imagen se haya cargado correctamente.
27  if imagen is None:
28
29      # Si no se ha encontrado la imagen, mostramos un mensaje.
30      print("ERROR: No se ha podido cargar la imagen.")
31
```

```

else:

    # CONVERTIMOS LA IMAGEN DE BGR A RGB

    # OpenCV carga las imágenes utilizando el orden de colores
    # BGR (Blue, Green, Red).

    # Tesseract trabaja con imágenes en formato RGB
    # (Red, Green, Blue).

    # Por eso hacemos esta conversión.
    imagen_rgb = cv2.cvtColor(
        imagen,
        cv2.COLOR_BGR2RGB
    )

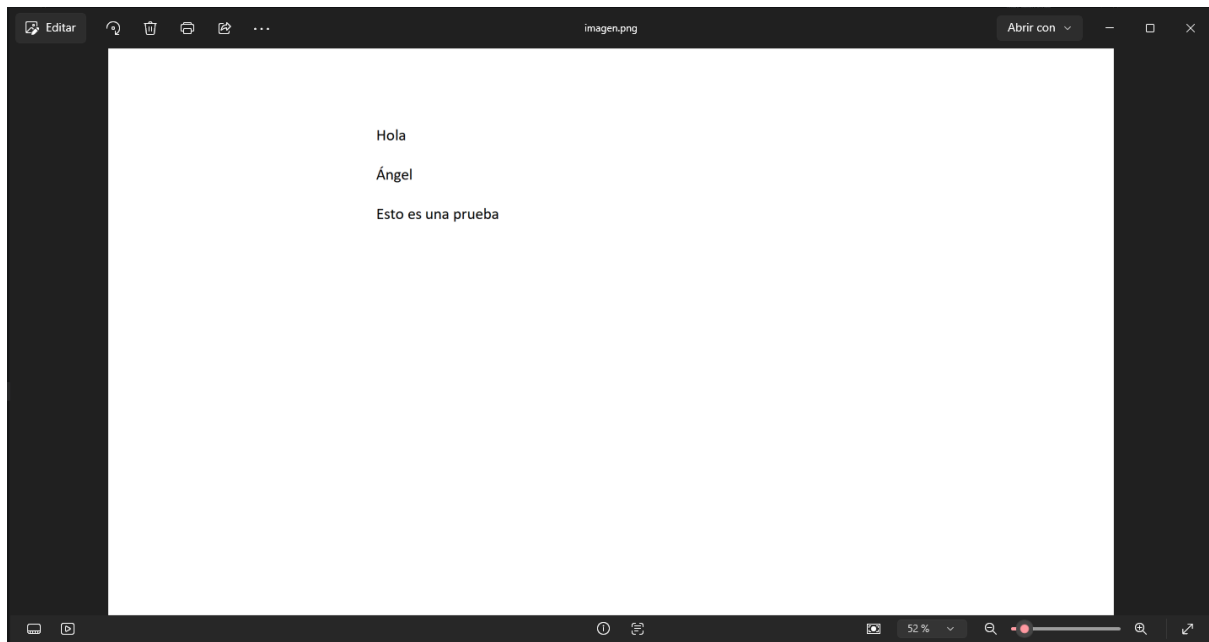
    # RECONOCIMIENTO DEL TEXTO

    # image_to_string() analiza la imagen mediante OCR
    # y devuelve el texto que ha conseguido reconocer.
    texto = pytesseract.image_to_string(
        imagen_rgb
    )

    # MOSTRAMOS EL RESULTADO

    # Mostramos en la terminal el texto reconocido.
    print("Texto reconocido:")
    print("-----")
    print(texto)

```



Resultado:

```
Texto reconocido:
-----
Hola
Angel

Esto es una prueba

PS C:\Users\angel\Desktop\SGEntregar>
```

Actividad 13

```
# Actividad 13

# Importamos la biblioteca requests.
# Nos permite hacer peticiones HTTP a una API.
import requests

# PEDIMOS LA ESPECIE AL USUARIO

# Pedimos al usuario que escriba una especie.
especie = input("Introduce una especie (Human, Alien, Robot): ")

# URL DE LA API

# Esta es la dirección de la API de Rick y Morty.
url = "https://rickandmortyapi.com/api/character"

# HACEMOS LA PETICIÓN A LA API

# Realizamos una petición GET.

# El parámetro "species" permite filtrar los personajes
# según la especie que ha introducido el usuario.
respuesta = requests.get(
    url,
    params={"species": especie}
)
```

```

# COMPROBAMOS SI LA PETICIÓN HA FUNCIONADO

# Si la API responde con código 200, la petición
# se ha realizado correctamente.
if respuesta.status_code == 200:

    # Convertimos la respuesta JSON en un diccionario
    # de Python.
    datos = respuesta.json()

    # Mostramos un título.
    print("\nPersonajes de la especie:", especie)
    print("-----")

    # Recorremos la lista de personajes.
    for personaje in datos["results"]:

        # Mostramos solamente el nombre del personaje.
        print(personaje["name"])

else:

    # Si la API no encuentra personajes de esa especie,
    # mostramos un mensaje de error.
    print("No se han encontrado personajes de esa especie.")

```

Resultado:

```

Personajes de la especie: Human
-----
Rick Sanchez
Morty Smith
Summer Smith
Beth Smith
Jerry Smith
Abradolf Lincler
Adjudicator Rick
Agency Director

```

Hay muchos más nombres, pero no merece la pena seguir mostrando para probar el punto.

Actividad 14

```
# Actividad 14

def es_segura(tablero, fila, columna):
    """
    Comprueba si podemos colocar una reina en la posición
    indicada sin que ataque a ninguna reina anterior.
    """

    # Recorremos las filas anteriores.
    for fila_anterior in range(fila):

        # Columna donde está colocada la reina anterior.
        columna_anterior = tablero[fila_anterior]

        # Comprobamos si están en la misma columna.
        if columna_anterior == columna:
            return False

        # Comprobamos si están en la misma diagonal.
        if abs(columna_anterior - columna) == abs(fila_anterior - fila):
            return False

    # Si no hay ningún conflicto, la posición es válida.
    return True

def contar_soluciones(tablero, fila, n):
    """
    Coloca las reinas fila por fila y devuelve
    el número total de soluciones encontradas.
    """
```

```

# Si hemos colocado todas las reinas,
# hemos encontrado una solución.
if fila == n:
    return 1

# Empezamos con 0 soluciones.
soluciones = 0

# Probamos todas las columnas de esta fila.
for columna in range(n):

    # Comprobamos si podemos colocar una reina aquí.
    if es_segura(tablero, fila, columna):

        # Colocamos la reina.
        tablero[fila] = columna

        # Intentamos colocar la siguiente reina.
        soluciones += contar_soluciones(tablero, fila + 1, n)

        # Volvemos atrás para probar otra posición.
        tablero[fila] = -1

# Devolvemos el número de soluciones encontradas.
return soluciones

# Pedimos el tamaño del tablero.
n = int(input("Introduce el tamaño N del tablero (4-10): "))

```

```

# Comprobamos que N está dentro del rango permitido.
if n < 4 or n > 10:
    print("ERROR: N debe estar entre 4 y 10.")
else:
    # Creamos el tablero.

    # Cada posición representa una fila.
    # El valor indica en qué columna está la reina.

    # Por ejemplo:
    # [1, 3, 0, 2]

    # significa que:
    # fila 0 -> columna 1
    # fila 1 -> columna 3
    # fila 2 -> columna 0
    # fila 3 -> columna 2
    tablero = [-1] * n

    # Calculamos las soluciones.
    total = contar_soluciones(tablero, 0, n)

    # Mostramos el resultado.
    print("Número total de soluciones:", total)

```

Resultado:

```

Introduce el tamaño N del tablero (4-10): 10
Número total de soluciones: 724
PS C:\Users\angel\Desktop\SGEentregar>

```

Actividad 15

```
1  # ACTIVIDAD 15 - FILTRO DEL PORTAPAPELES
2
3  import sys
4  import time
5  import pyperclip
6
7
8  # Comprobamos que se ha indicado el fichero de palabras
9  # prohibidas como parámetro.
10
11 if len(sys.argv) != 2:
12     print("Uso: python UD00Act2.15AngelEsteban.py lista.txt")
13
14 else:
15     # Guardamos el nombre del fichero.
16
17     fichero = sys.argv[1]
18
19     # Leemos el fichero de palabras prohibidas.
20
21     palabras_prohibidas = []
22
23     with open(fichero, "r", encoding="utf-8") as archivo:
24
25         for linea in archivo:
26
27             palabra = linea.strip()
28
29             if palabra != "":
30                 palabras_prohibidas.append(palabra)
```

```

# Mostramos las palabras que hemos cargado.

print("Filtro iniciado.")
print("Palabras prohibidas:")

for palabra in palabras_prohibidas:
    print("-", palabra)

print()
print("Ahora copia cualquier texto.")
print("Pulsa Ctrl+C en esta ventana para terminar.")
print()

# Guardamos el contenido actual del portapapeles.

texto_anterior = pyperclip.paste()

# El programa se queda funcionando continuamente.

try:
    while True:
        # Miramos el contenido actual del portapapeles.
        texto = pyperclip.paste()

        # Si no ha cambiado, esperamos medio segundo.
        if texto == texto_anterior:
            time.sleep(0.5)
            continue

```

```
# Guardamos el nuevo contenido.
texto_anterior = texto

# Empezamos a filtrar el texto.

texto_filtrado = texto

# Recorremos todas las palabras prohibidas.
for palabra in palabras_prohibidas:

    # Versión en minúsculas para comparar
    # sin distinguir mayúsculas/minúsculas.
    palabra_minusculas = palabra.lower()

    resultado = ""
    posicion = 0

    # Recorremos el texto.
    while posicion < len(texto_filtrado):

        # Extraemos un trozo del mismo tamaño
        # que la palabra prohibida.
        trozo = texto_filtrado[
            posicion:posicion + len(palabra)
        ]

        # Comparamos sin distinguir mayúsculas
        # y minúsculas.
        if trozo.lower() == palabra_minusculas:

            # Sustituimos la palabra por asteriscos.
            resultado += "*" * len(palabra)
```

```

        # Saltamos los caracteres de la palabra.
        posicion += len(palabra)

    else:

        # Copiamos el carácter normal.
        resultado += texto_filtrado[posicion]

        posicion += 1

# Actualizamos el texto filtrado.
texto_filtrado = resultado

# Si hemos encontrado alguna palabra prohibida,
# copiamos el texto modificado al portapapeles.

if texto_filtrado != texto:

    pyperclip.copy(texto_filtrado)

    print("Texto filtrado:")
    print(texto_filtrado)
    print()

else:

    print("Texto copiado sin palabras prohibidas.")
    print()

```

```

except KeyboardInterrupt:

    print()
    print("Filtro detenido.")

```

lista.txt

```

1  Hola
2  pYtHon

```

Resultado:

```
PS C:\Users\angel\Desktop\SGEentregar> python UD00Act2.15AngelEsteban.py lista.txt
Filtro iniciado.
Palabras prohibidas:
- Hola
- pYtHon

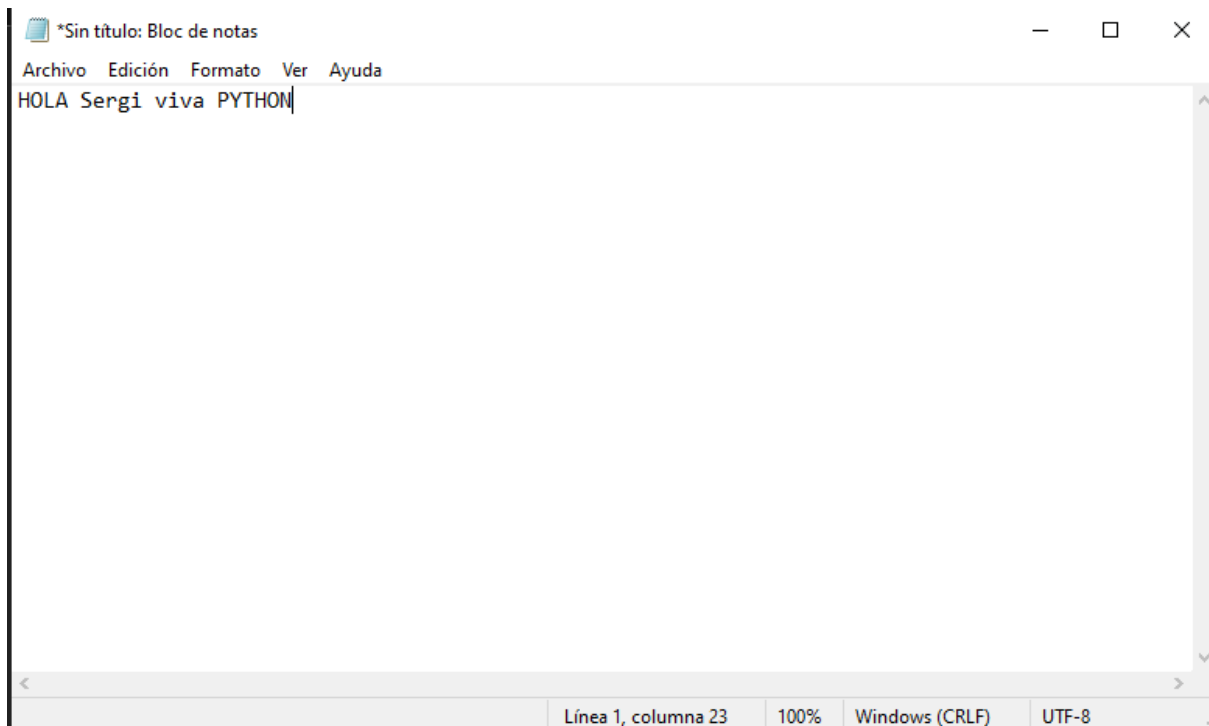
Ahora copia cualquier texto.
Pulsa Ctrl+C en esta ventana para terminar.

Texto filtrado:
**** Sergi viva *****
```

Texto copiado sin palabras prohibidas.

Filtro detenido.

```
PS C:\Users\angel\Desktop\SGEentregar> █
```



Primero ejecutamos el programa y nos vamos al bloc de notas y copiamos la frase que queremos filtrar. Tras eso, el programa detecta un cambio en el portapapeles y filtra lo que hay y te muestra cómo lo ha filtrado. Finalmente, te mete en el portapapeles las nuevas palabras filtradas y las vuelve a analizar

(por eso sale “Texto copiado sin palabras prohibidas”). Cuando le das Control + C en la consola, se acaba el programa.

Enlace GitHub

<https://github.com/angelesteeban/SGE2DAMAngelEspan>